Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(МОСКОВСКИЙ ПОЛИТЕХ)

КУРСОВОЙ ПРОЕКТ
по теме:

ВЕБ-ПРИЛОЖЕНИЕ ДЛЯ РОЗНИЧНОЙ СЕТИ СУПЕРМАРКЕТОВ

по курсу Программная инженерия

по направлению 09.03.03 – Прикладная информатика
Образовательная программа (профиль)
«Корпоративные информационные системы»

Студент: Андриянова А.А., 231-361

Преподаватель: Пардаев А.А.

Москва 2026

ЗАДАНИЕ НА КУРСОВОЕ ПРОЕКТИРОВАНИЕ

Целью выполнения курсового проекта является закрепление и углубление теоретических знаний и практических навыков, полученных при изучении дисциплины «Программная инженерия», а также приобретение навыков проектирования и разработки информационных систем и/или их модулей, публичной демонстрации и презентации результатов курсовой работы при ее защите.

В ходе выполнения курсовой работы необходимо спроектировать и реализовать информационную систему (модуль системы, программный продукт), позволяющую добавлять/модифицировать/удалять данные, выполнять запросы к данным, импортировать и экспортировать данные в заранее предопределенных форматах. Тематика работы выбирается на основании проведенного исследования в ходе лабораторного практикума по данной дисциплине.

Для реализации информационной системы необходимо использовать навыки, полученные при изучении дисциплин учебного плана. В работе необходимо представить обоснование выбора технологических решений и применяемых инструментальных средств реализации информационной системы.

Вариант задания: Розничная сеть супермаркетов

Бизнес-процесс: управление запасами и логистикой

Основные требования к информационной системе:

1. Автоматический учет остатков по каждому магазину и складу в режиме реального времени, обеспечивающий актуальную информацию о наличии товаров.

2. Прогнозирование потребностей на основе анализа статистики продаж для оптимизации закупок и предотвращения дефицита или избытка товаров.

3. Интеграция с поставщиками для электронного заказа товаров, автоматизация процесса формирования и отправки заявок.

4. Отслеживание сроков годности товаров и управление списанием просроченной продукции, включая систему оповещений об истекающих сроках.

5. Дашборды для руководителей, отображающие ключевые показатели: текущие запасы, оборачиваемость товаров, прибыль по категориям и другие аналитические данные.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 5
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ 7
1.1 Категории пользователей 7
1.2 Перечень сущностей и взаимосвязей 8
1.3 Анализ аналогов 10
1.4 Спецификация требований 11
2 ПРОЕКТИРОВАНИЕ 13
2.1 Архитектура системы 13
2.2 Модель вариантов использования 13
2.3 Проектирование классов и подсистем 14
2.4 Структура базы данных 17
2.5 Обоснование технологического стека 18
3 РЕАЛИЗАЦИЯ ИНФОРМАЦИОННОЙ СИСТЕМЫ 19
3.1 Реализация серверной части 19
3.2 Реализация клиентской части 19
3.3 Обработка ошибок и валидация 19
4 ПРАКТИЧЕСКИЕ РЕЗУЛЬТАТЫ 21
4.1 Тестовые планы и сценарии 21
4.2 Оценка соответствия требованиям 24
4.3 Перспективы развития 24
ЗАКЛЮЧЕНИЕ 26
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 27
ПРИЛОЖЕНИЕ 28
Приложение А – ER-диаграмма базы данных 28
Приложение Б – Контекстная диаграмма 28
Приложение В –Диаграмма контейнеров 29
Приложение Г – SQL-описание структуры базы данных 29
Приложение Д – Словарь данных (таблицы) 33
Приложение Е – Пояснение к диаграммам проектирования 45

ВВЕДЕНИЕ

Современные розничные сети сталкиваются с вызовом эффективного управления товарными запасами в условиях многозвенной логистической цепи «поставщик – центральный склад – распределительные центры – магазины». Рост конкуренции требует от ритейлеров не только широкого ассортимента, но и минимизации издержек, связанных с хранением, транспортировкой и списанием товаров. Особую остроту приобретает проблема учета скоропортящейся продукции и товаров с ограниченным сроком годности, где своевременное выявление просрочки напрямую влияет на финансовые результаты.

Актуальность разработки информационной системы управления запасами для розничной сети супермаркетов обусловлена необходимостью автоматизации ключевых процессов: учета остатков в реальном времени, прогнозирования потребностей на основе статистики продаж, интеграции с поставщиками для электронного заказа и контроля сроков годности. Централизованное хранение данных, разграничение прав доступа по ролям и наличие аналитических дашбордов позволяют повысить прозрачность бизнес-процессов и снизить влияние человеческого фактора.

Степень разработанности темы характеризуется наличием на рынке мощных ERP-решений, таких как «1С:Управление торговлей» и SAP S/4HANA [1, 2]. Однако данные продукты обладают высокой стоимостью внедрения, сложностью кастомизации и часто избыточны для среднего бизнеса, требуя длительной адаптации под конкретные нужды предприятия. Таким образом, существует потребность в специализированном решении, сочетающем простоту внедрения, современный технологический стек и целевую ориентацию на управление товарными запасами.

Целью курсового проекта является проектирование и реализация информационной системы «Управление запасами розничной сети»,

обеспечивающей автоматизацию учета товаров, прогнозирования спроса, взаимодействия с поставщиками и контроля сроков годности.

Для достижения поставленной цели решаются следующие задачи:

1. выполнить анализ предметной области и определить категории пользователей;
2. выделить ключевые сущности и их взаимосвязи;
3. провести анализ существующих аналогов и сформировать спецификацию требований;
4. обосновать архитектурное решение и выбрать технологический стек;
5. спроектировать структуру базы данных и выполнить её нормализацию;
6. реализовать серверную и клиентскую части приложения;
7. разработать тестовые сценарии и провести проверку корректности работы;
8. сформулировать перспективы дальнейшего развития системы.

Объектом исследования является процесс информационного сопровождения управления товарными запасами в розничной сети супермаркетов.

Предметом исследования являются методы и средства автоматизации операций учёта, прогнозирования, заказа товаров и контроля сроков годности с применением веб-ориентированной информационной системы.

Практическим результатом работы является функционирующий прототип веб-приложения на базе Spring Boot и MySQL, поддерживающий ролевую модель (администратор, менеджер, кладовщик), CRUD-операции над основными сущностями, формирование прогнозов и генерацию тестовых данных для демонстрации возможностей.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Категории пользователей

В проектируемой системе управления запасами розничной сети выделены три роли пользователей, каждая из которых обладает определённым набором прав доступа к функционалу. Разграничение прав реализуется на уровне серверной части с использованием механизмов ролевой авторизации Spring Security. В классе-сущности Role, расположенном в пакете data.entities, определены возможные роли в виде перечисления (enum).

Роль «Кладовщик» (STOREKEEPER) предназначена для сотрудников, непосредственно работающих с товародвижением на складах и в магазинах. Кладовщик выполняет операции приёма товаров по накладным, перемещения между складами, списания просроченной или бракованной продукции, а также проводит инвентаризацию. Данная роль не имеет доступа к аналитическим отчётам и управлению справочниками.

Роль «Менеджер» (MANAGER) предназначена для сотрудников, отвечающих за планирование закупок и анализ эффективности. Менеджер имеет доступ к истории продаж, может просматривать текущие остатки, формировать прогнозы потребности на основе статистики, создавать заказы поставщикам и корректировать их. Кроме того, менеджер может добавлять новых пользователей (кроме администраторов) и редактировать их данные.

Роль «Администратор системы» (ADMIN) обладает полными правами на управление всеми компонентами системы. Администратор управляет пользователями (создание, блокировка, смена ролей, удаление), ведёт справочники (товары, категории, поставщики, склады), настраивает параметры системы (алгоритмы прогнозирования, пороговые значения уведомлений), а также имеет доступ ко всем отчётам и дашбордам.

Сводное описание ролей, их назначения и основных функциональных операций представлено в таблице 1.

Роль	Назначение	Основные операции
Кладовщик	Операционная работа с товарами на складе	Приёмка товаров; Перемещение между складами; Списание товаров; Инвентаризация
Менеджер	Планирование закупок и аналитика	Просмотр остатков; Прогнозирование спроса; Создание заказов поставщикам; Просмотр дашбордов; Управление пользователями (кроме администраторов)
Администратор	Управление системой и справочниками	Ведение справочников (товары, категории, поставщики, склады); Управление пользователями; Настройка параметров системы; Полный доступ к отчётам

Таблица 1 – Роли пользователей и доступный функционал

1.2 Перечень сущностей и взаимосвязей

На основе анализа функциональных требований выделены ключевые сущности предметной области и определены связи между ними. Все сущности реализованы как JPA-классы в пакете data.entities.

Основные сущности системы:

Product (Товар) – центральная сущность, содержащая информацию о товарах: наименование, артикул (SKU), штрихкод, единица измерения, цены закупки и продажи, срок годности (для скоропортящихся товаров).

Category (Категория) – иерархический справочник для группировки товаров.

Warehouse (Склад/Магазин) – места хранения товаров: центральные склады, распределительные центры, торговые залы магазинов.

Supplier (Поставщик) – контрагенты, поставляющие товары.

Inventory (Остатки) – учёт количества товаров на конкретном складе в реальном времени.

StockMovement (Движение товаров) – журнал всех операций с товарами (приход, расход, перемещение, списание).

Batch (Партия) – учёт товаров по партиям для контроля сроков годности.

Order (Заказ поставщику) – документы заказа товаров у поставщиков.

User (Пользователь) – учётные записи сотрудников с привязкой к роли (Role).

Детальное описание связей:

Связи справочников:

Product (M) – (1) Category: многие товары принадлежат одной категории.

Category (1) – (M) Category: иерархическая связь категорий (родитель-потомок).

Связи учёта остатков:

Product (1) – (M) Inventory: один товар может храниться на нескольких складах.

Warehouse (1) – (M) Inventory: на одном складе может храниться много товаров.

Product (1) – (M) Batch: один товар может поступать разными партиями.

Warehouse (1) – (M) Batch: на одном складе может храниться много партий.

Связи движения товаров:

Product (1) – (M) StockMovement: по одному товару выполняется много операций.

Warehouse (1) – (M) StockMovement: склад может быть источником или получателем перемещения.

User (1) – (M) StockMovement: пользователь проводит много операций.

Batch (1) – (M) StockMovement: одна партия может участвовать в нескольких операциях.

Связи управления заказами:

Supplier (1) – (M) Order: один поставщик может иметь много заказов.

User (1) – (M) Order: один пользователь создаёт много заказов.

1.3 Анализ аналогов

Для обоснования целесообразности разработки выполнен сравнительный анализ существующих на рынке решений, автоматизирующих управление товарными запасами.

В качестве аналогов рассмотрены:

«1С:Управление торговлей» – флагманское решение на рынке СНГ для автоматизации торговых операций.

SAP S/4HANA – мировое enterprise-решение для управления цепями поставок.

Вывод: целесообразна разработка специализированной системы, которая не пытается конкурировать с универсальными платформами в их всеохватности, а предлагает «острый» инструмент, идеально решающий задачу управления запасами – быстрее, проще и дешевле.

Сравнительная характеристика оценки аналогов системы представлена в таблице 2.

Критерий	1С:Управление торговлей	SAP S/4HANA	Разрабатываемая система
Фокус функционала	Универсальная ERP	Управление цепями поставок	Узкая специализация на запасах

Критерий	1С:Управление торговлей	SAP S/4HANA	Разрабатываемая система
Сложность внедрения	Высокая	Очень высокая	Низкая
Время внедрения	Месяцы	Годы	Недели (MVP за 3 мес.)
Технологии	Собственная платформа	Проприетарные	Современные (Java, Spring)
Стоимость владения	Высокая	Очень высокая	Низкая и предсказуемая
Кастомизация	Сложная и дорогая	Сложная	Высокая, заложена в проект

Таблица 2 – Сравнительная оценка аналогов

1.4 Спецификация требований

Требования к системе сформированы на основе анализа бизнес-процессов розничной сети и разделены на функциональные и нефункциональные.

Функциональные и нефункциональные требования к системе представлены в таблицах 3 и 4 соответственно.

ID	Требование	Приоритет
Учёт остатков		

ID	Требование	Приоритет
ФТ-001	Ведение базы данных товаров (CRUD)	Must-Have
ФТ-002	Отслеживание остатков в реальном времени по складам	Must-Have
ФТ-006	Учёт партий и сроков годности	Must-Have
ФТ-007	Автоматическое оповещение о просрочке	Must-Have

Таблица 3 – Функциональные требования

ID	Требование	Метрика/Пояснение
НФТ-001	Время отклика интерфейса	Не более 2 секунд для типовых операций
НФТ-002	Безопасность доступа	Аутентификация; авторизация по ролям;
НФТ-003	Надёжность хранения	Транзакционность операций

Таблица 4 – Нефункциональные требования

2 ПРОЕКТИРОВАНИЕ

2.1 Архитектура системы

Система спроектирована как веб-приложение, построенное по классической трёхуровневой архитектуре клиент-сервер с логическим разделением на слои. Данный подход обеспечивает простоту развёртывания и прозрачность трассировки запросов.

Клиентский уровень:

- веб-браузеры (Chrome, Firefox, Edge);
- соединение: HTTPS.

Серверный уровень:

- веб-сервер (встроенный Tomcat в Spring Boot);
- сервер приложений (Spring Boot).

Уровень данных:

- система управления базами данных: MySQL;
- соединение: JDBC.

Ключевые характеристики:

- безопасность: HTTPS для клиент-серверного взаимодействия, CSRF-защита;
- технологии: Java 21, Spring Boot 3, Spring MVC, Spring Security, Spring Data JPA, MySQL, Thymeleaf, Bootstrap;
- масштабируемость: Разделение на уровни позволяет масштабировать компоненты независимо.

2.2 Модель вариантов использования

Модель вариантов использования отражает функциональные требования к системе и взаимодействие пользователей различных ролей с её компонентами.

Актёры системы:

1. Кладовщик (STOREKEEPER) – выполняет операционные задачи по движению товаров

2. Менеджер (MANAGER) – осуществляет планирование закупок и анализ

3. Администратор (ADMIN) – управляет системой и справочниками

Основные варианты использования:

1. Управление учётными записями (Администратор) – реализовано в UserController и UserService.

2. Ведение справочников (Администратор) – ProductController, SupplierController, WarehouseController.

3. Управление заказами поставщикам (Менеджер) – OrderController (в перспективе).

4. Прогнозирование потребности (Менеджер) – модуль Analytics (в перспективе).

5. Просмотр дашбордов (Менеджер, Администратор) – PagesController.

6. Приёмка, перемещение и списание товаров (Кладовщик) – StockMovementController и BatchController.

2.3 Проектирование классов и подсистем

Проектирование выполнено с применением объектно-ориентированного подхода и многослойной архитектуры, которая обеспечивает разделение ответственности между различными уровнями приложения. Все классы сгруппированы в пакеты в соответствии с их ролью в архитектуре.

Архитектурные слои:

Слой представления (Presentation Layer): Реализован с помощью контроллеров (пакет controllers). Они обрабатывают HTTP-запросы, взаимодействуют с сервисным слоем и возвращают клиенту данные в формате HTML-шаблона (для MVC). Контроллеры используют DTO (Data Transfer Objects) из пакета data.dtos для обмена данными, чтобы не раскрывать внутреннюю структуру сущностей.

Пример: SupplierController обрабатывает запросы по адресу /api/suppliers и возвращает список поставщиков в виде SupplierDto.

Слой бизнес-логики (Business Logic Layer): Содержит сервисы (пакет buiseness). Здесь инкапсулирована вся логика приложения. Для связанности каждый сервис реализует соответствующий интерфейс (например, ProductService реализует ProductServiceInterface).

Пример: SupplierService реализует метод getAllSuppliers(), обращаясь к репозиторию и преобразуя полученные сущности Supplier в SupplierDto.

Слой доступа к данным (Data Access Layer): Представлен репозиториями (пакет data.repositories), которые наследуют функциональность JpaRepository от Spring Data JPA. Они предоставляют стандартные методы для CRUD-операций и позволяют определять пользовательские запросы к базе данных.

Пример: SupplierRepository расширяет JpaRepository<Supplier, Long> и предоставляет методы findAll(), save() и др.

Доменный слой (Domain Layer): Состоит из JPA-сущностей (пакет data.entities), которые являются объектным отражением таблиц базы данных.

Пример: Класс Supplier аннотирован @Entity и содержит поля, соответствующие колонкам таблицы supplier.

Выделенные подсистемы (модули):

Исходя из структуры пакетов, в системе можно выделить следующие функциональные подсистемы:

1. Подсистема управления товарами: ProductController, ProductService(Interface), ProductRepository, сущность Product, а также связанные Category и ProductDto.
2. Подсистема управления складскими запасами: InventoryController, InventoryService, InventoryRepository,

сущность Inventory, а также множество DTO для детализации (InventoryDetailDto, InventoryStatsDto и др.).

3. Подсистема управления поставщиками: SupplierController, SupplierService(Interface), SupplierRepository, сущность Supplier и SupplierDto.

4. Подсистема управления партиями и сроками годности: BatchController, BatchService(Interface), BatchRepository, сущность Batch и соответствующие DTO (BatchDto, BatchFilterDto).

5. Подсистема учета движений товаров: StockMovementController, StockMovementService(Interface), StockMovementRepository, сущность StockMovement и StockMovementDto.

6. Подсистема управления пользователями и безопасностью: UserController, UserService, UserRepository, сущности User и Role, а также MyUserDetailService для интеграции со Spring Security. Конфигурация безопасности вынесена в пакет config (SecurityConfig).

7. Подсистема навигации: PagesController для отображения HTML-страниц и ProfileController для управления профилем пользователя.

Взаимодействие слоев:

Взаимодействие между слоями организовано следующим образом:

Контроллер принимает запрос и вызывает метод Сервиса (через его интерфейс).

Сервис реализует бизнес-логику, при необходимости обращаясь к Репозиторию для получения или сохранения данных.

Репозиторий возвращает Сущность, которую Сервис может преобразовать в DTO.

Сервис возвращает результат (сущность или DTO) обратно в Контроллер.

Диаграмма классов представлена на рисунке 1.

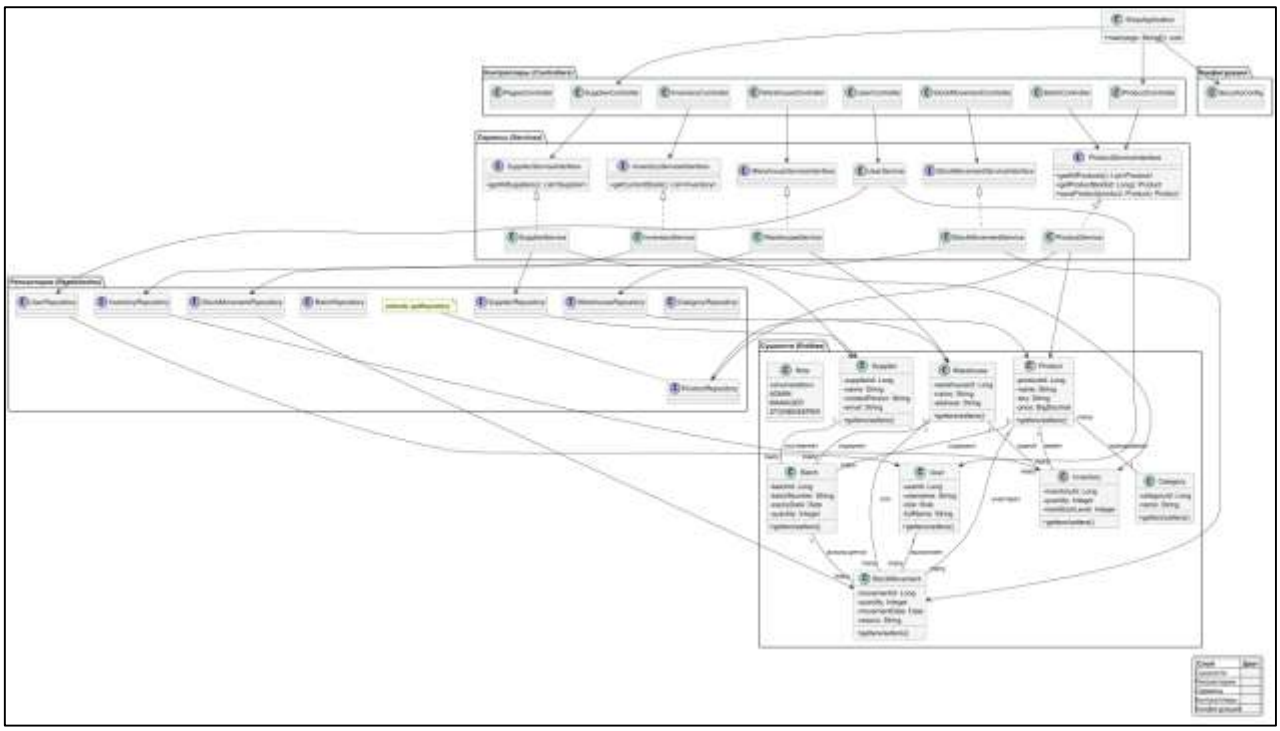

Рисунок 1 – UML-диаграмма классов

2.4 Структура базы данных

Структура базы данных спроектирована на основе выделенных сущностей и связей между ними. При проектировании применялись принципы нормализации для исключения избыточности и обеспечения целостности данных.

Ключевые таблицы:

- product – товары;
- category – категории товаров;
- warehouse – склады и магазины;
- supplier – поставщики;
- inventory – остатки товаров;
- stock_movement – движения товаров;
- batch – партии товаров;
- orders – заказы поставщикам;
- user – пользователи системы.

В приложении Б представлена ER-диаграмма, в приложении В – словарь данных, в приложении Г – SQL-скрипты создания таблиц.

2.5 Обоснование технологического стека

Выбор технологического стека определялся требованиями к надёжности, производительности, доступности инструментов и поддержке учебного процесса.

Серверная часть:

– Java 21 – современная версия языка с долгосрочной поддержкой;

– Spring Boot 3 – фреймворк для быстрого создания production-grade приложений;

– Spring MVC – реализация шаблона Model-View-Controller;

– Spring Security – обеспечение аутентификации, авторизации и защиты;

– Spring Data JPA – упрощение доступа к данным.

База данных:

– MySQL – надёжная, производительная и широко распространённая реляционная СУБД.

Клиентская часть:

– Bootstrap 5 – CSS-фреймворк для адаптивной вёрстки;

– JavaScript – обеспечение интерактивности на стороне клиента.

Инструменты разработки:

– IntelliJ IDEA – интегрированная среда разработки;

– Maven – система сборки и управления зависимостями;

– Git – система контроля версий.

Данный стек обеспечивает соответствие всем нефункциональным требованиям и позволяет получить работающий прототип в короткие сроки.

3 РЕАЛИЗАЦИЯ ИНФОРМАЦИОННОЙ СИСТЕМЫ

3.1 Реализация серверной части

Серверная часть реализована на базе Spring Boot и обеспечивает обработку HTTP-запросов, выполнение бизнес-логики и взаимодействие с базой данных. Структура проекта полностью соответствует архитектуре, описанной в разделе 2.3.

3.2 Реализация клиентской части

Клиентская часть реализована в виде набора HTML-шаблонов Thymeleaf с использованием Bootstrap 5 для адаптивной вёрстки и JavaScript для обеспечения интерактивности.

Основные страницы интерфейса:

1. /login – страница аутентификации
2. /main – главная страница с дашбордом
3. /products – управление товарами
4. /warehouses – управление складами
5. /batches – управление партиями
6. /adminPanel– управление пользователями (только ADMIN)

Навигация:

На всех страницах присутствует общая панель навигации (navbar), загружаемая через JavaScript (/js/navbar.js). Она динамически формируется на основе роли авторизованного пользователя, получаемой с сервера через эндпоинт /api/user/current.

3.3 Обработка ошибок и валидация

На стороне сервера:

Используется CustomErrorController для обработки ошибок HTTP (404, 500 и др.) и отображения пользовательских страниц ошибок.

Валидация входных данных осуществляется с помощью стандартных средств Spring в контроллерах.

Для защиты от атак используются токены, встроенные в Spring Security.

На стороне клиента:

Реализована базовая валидация форм с помощью HTML5-атрибутов (required, type="email").

При ошибках аутентификации пользователь перенаправляется на страницу логина с сообщением об ошибке.

При неверном формате или отсутствии данных в AJAX-запросах пользователю выводится уведомление с помощью функции showNotification().

4 ПРАКТИЧЕСКИЕ РЕЗУЛЬТАТЫ

4.1 Тестовые планы и сценарии

Тестирование выполнено на уровне системных сценариев с проверкой ключевых пользовательских потоков. Использовались следующие виды тестирования: функциональное (проверка соответствия требованиям), интеграционное (проверка взаимодействия веб-слоя и БД), а также проверка требований безопасности (доступ по ролям).

В таблицах 5 и 6 представлены позитивные и негативные тестовые сценарии для роли «Администратор» и роли «Кладовщик».

ID	Предусловия	Действия	Ожидаемый результат	Статус
ADM-P1	Пользователь с ролью ADMIN авторизован в системе	Перейти на страницу /admin/users	Отображается список всех пользователей системы с возможностью редактирования	Выполнен
ADM-P2	Администратор находится на странице управления пользователями	Нажать кнопку «Создать пользователя», заполнить форму (логин, пароль, роль) и сохранить	В списке пользователей появляется новая запись; данные сохранены в БД	Выполнен
ADM-P3	В системе есть тестовые данные	Перейти на страницу тестирования (/test) и выбрать генерацию 10 поставщиков	Через несколько секунд появляется	Выполнен

ID	Предусловия	Действия	Ожидаемый результат	Статус
			уведомление об успехе; в таблице поставщиков отображается 10 новых записей	
ADM-P4	Администратор открыл страницу товаров	Нажать «Добавить товар», заполнить обязательные поля (название, артикул, цена)	Товар появляется в общем списке; данные корректно сохраняются	Выполнен
STK-P1	Поступила новая партия товара	Открыть страницу /batches, нажать «Создать партию», заполнить данные (товар, поставщик, количество, срок годности)	Партия сохраняется; количество товара на складе увеличивается	Выполнен
STK-P2	У товара истёк срок годности	На странице партий нажать кнопку	Партия помечается как списанная;	Выполнен

ID	Предусловия	Действия	Ожидаемый результат	Статус
		«Списать» для просроченной партии	количество товара на складе уменьшается	

Таблица 5 – Позитивные тестовые сценарии

ID	Предусловия	Действия	Ожидаемый результат	Статус
ADM-N1	Администратор создаёт нового пользователя	Оставить поле «логин» пустым и нажать «Сохранить»	Форма не отправляется; появляется сообщение об обязательном поле	Выполнен
ADM-N2	Администратор пытается понизить свою роль до кладовщика	В разделе редактирования своего профиля выбрать роль STOREKEEPER	Система не позволяет изменить собственную роль или выдаёт предупреждение	Выполнен
STK-N1	Кладовщик пытается открыть страницу	Ввести в адресной строке /admin/users	Доступ запрещён (HTTP 403); происходит	Выполнен

ID	Предусловия	Действия	Ожидаемый результат	Статус
	управления пользователями		перенаправление на главную или страницу ошибки	
STK-N2	Кладовщик при списании указывает отрицательное количество	Ввести отрицательное число в поле «Количество»	Поле не принимает отрицательные значения (валидация на стороне клиента)	Выполнен

Таблица 6 – Негативные тестовые сценарии

4.2 Оценка соответствия требованиям

Результаты тестирования показывают выполнение всех ключевых функциональных требований, обозначенных в разделе 1.4. Система позволяет вести справочники, отслеживать остатки, фиксировать движения товаров и управлять партиями.

Нефункциональные требования, связанные с безопасностью (аутентификация, авторизация, CSRF), полностью обеспечены средствами Spring Security. Требования к производительности (НФТ-001) в целом выполняются для текущего объёма данных.

4.3 Перспективы развития

Перспективы развития системы включают функциональные и технические улучшения.

Функциональные расширения:

1. Добавление модуля аналитики и прогнозирования спроса на основе истории продаж (SalesHistory), как того требует бизнес-процесс.
2. Реализация полноценного модуля заказов (Order и OrderItem) с возможностью автоматического формирования на основе прогнозов и отправки поставщикам по email.
3. Добавление отчётности по оборачиваемости товаров и прибыльности.

Технические улучшения:

1. Внедрение миграций базы данных (Liquibase или Flyway).
2. Реализация постраничной навигации (pagination) и серверной фильтрации для больших объёмов данных.
3. Контейнеризация приложения с помощью Docker для упрощения развёртывания.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсового проекта спроектирована и реализована информационная система «Управление запасами розничной сети», предназначенная для поддержки ключевых процессов товародвижения в супермаркете.

Выполнен анализ предметной области, определены категории пользователей (администратор, менеджер, кладовщик) и выделены основные сущности данных. Сформирована спецификация функциональных и нефункциональных требований, обоснован выбор архитектуры и технологического стека (Java 21, Spring Boot 3, MySQL).

Практическим результатом работы является функционирующий прототип веб-приложения. Серверная часть реализована в соответствии с многослойной архитектурой и включает в себя пакеты контроллеров (controllers), сервисов (buiseness), репозиториев (data.repositories), сущностей (data.entities) и DTO (data.dtos). Клиентская часть обеспечивает удобный интерфейс для выполнения основных операций: ведение справочников поставщиков, товаров и складов, учёт остатков, управление партиями и движениями товаров.

Проведённое тестирование подтвердило корректность работы основных пользовательских сценариев и механизмов безопасности. Определены направления дальнейшего развития системы, ориентированные на расширение аналитического функционала и повышение надёжности.

Таким образом, цель курсового проекта достигнута, все поставленные задачи решены.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Spring Boot Reference Documentation [Электронный ресурс]. – URL: https://docs.spring.io/spring-boot/docs/current/reference/html/ (дата обращения: 10.03.2025).
2. MySQL 8.0 Reference Manual [Электронный ресурс]. – URL: https://dev.mysql.com/doc/refman/8.0/en/ (дата обращения: 10.03.2025).
3. Thymeleaf Documentation [Электронный ресурс]. – URL: https://www.thymeleaf.org/documentation.html (дата обращения: 11.03.2025).
4. Буч Г., Рамбо Д., Якобсон А. Язык UML. Руководство пользователя. – М.: ДМК Пресс, 2007. – 496 с.
5. ГОСТ 7.32–2017. Система стандартов по информации, библиотечному и издательскому делу. Отчет о научно-исследовательской работе. Структура и правила оформления. – М.: Стандартинформ, 2017.

ПРИЛОЖЕНИЕ

Приложение А – ER-диаграмма базы данных

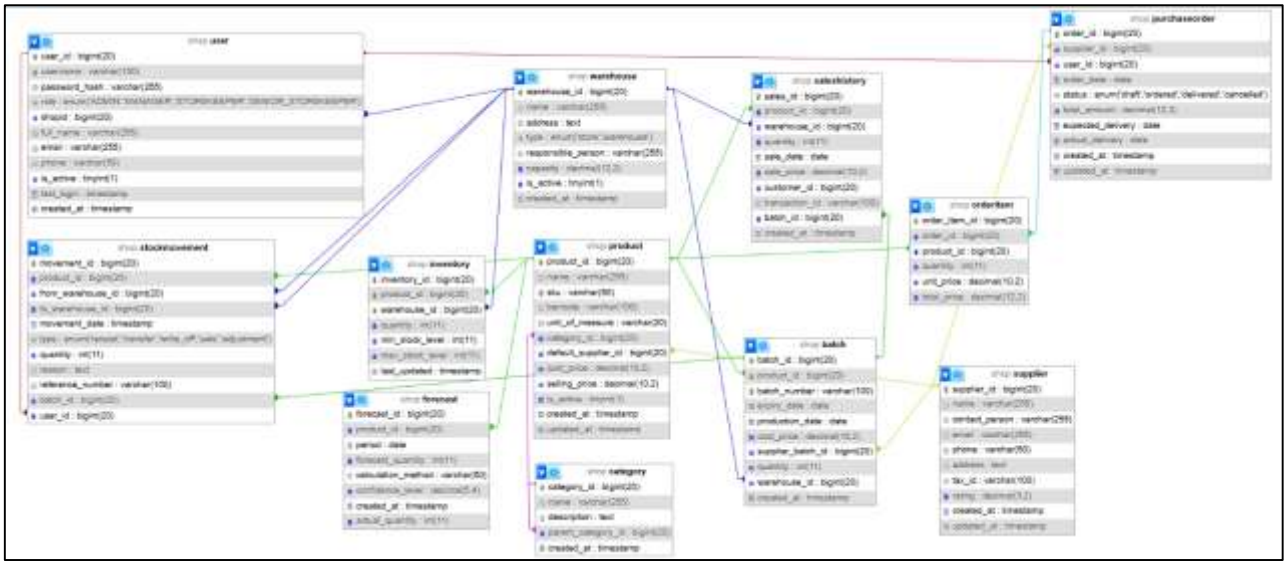

Рисунок А.1 – ER-диаграмма базы данных

Приложение Б – Контекстная диаграмма

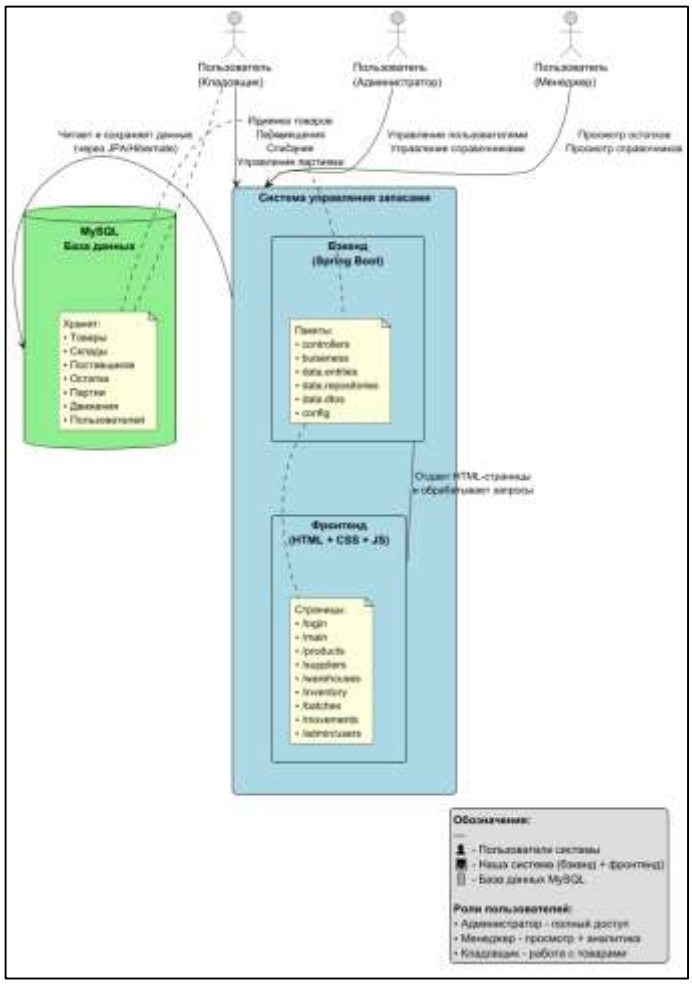

Рисунок Б.1 – Контекстная диаграмма

Приложение В –Диаграмма контейнеров

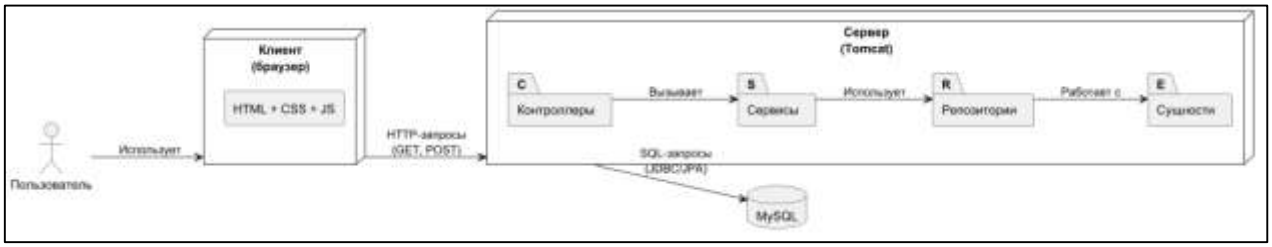

Рисунок В.1 – Диаграмма контейнеров

Приложение Г – SQL-описание структуры базы данных

```
CREATE DATABASE IF NOT EXISTS `shop`
CHARACTER SET utf8mb4
COLLATE utf8mb4_general_ci;
USE `shop`;
CREATE TABLE `category` (
  `category_id` bigint(20) NOT NULL AUTO_INCREMENT,
  `name` varchar(255) NOT NULL,
  `description` text DEFAULT NULL,
  `parent_category_id` bigint(20) DEFAULT NULL,
  `created_at` timestamp NOT NULL DEFAULT current_timestamp(),
  PRIMARY KEY (`category_id`),
  KEY `parent_category_id` (`parent_category_id`),
  CONSTRAINT `category_ibfk_1` FOREIGN KEY (`parent_category_id`)
   REFERENCES `category` (`category_id`) ON DELETE SET NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4
COLLATE=utf8mb4_general_ci;
CREATE TABLE `supplier` (
  `supplier_id` bigint(20) NOT NULL AUTO_INCREMENT,
  `name` varchar(255) NOT NULL,
  `contact_person` varchar(255) DEFAULT NULL,
  `email` varchar(255) DEFAULT NULL,
  `phone` varchar(50) DEFAULT NULL,
```

```
  `address` text DEFAULT NULL,
  `tax_id` varchar(100) DEFAULT NULL,
  `rating` decimal(3,2) DEFAULT NULL CHECK (`rating` >= 0 and `rating` <= 5),
  `created_at` timestamp NOT NULL DEFAULT current_timestamp(),
  `updated_at` timestamp NOT NULL DEFAULT current_timestamp() ON UPDATE current_timestamp(),
  PRIMARY KEY (`supplier_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
CREATE TABLE `warehouse` (
  `warehouse_id` bigint(20) NOT NULL AUTO_INCREMENT,
  `name` varchar(255) NOT NULL,
  `address` text DEFAULT NULL,
  `type` enum('store','warehouse') NOT NULL,
  `responsible_person` varchar(255) DEFAULT NULL,
  `capacity` decimal(12,2) DEFAULT NULL,
  `is_active` tinyint(1) DEFAULT 1,
  `created_at` timestamp NOT NULL DEFAULT current_timestamp(),
  PRIMARY KEY (`warehouse_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
CREATE TABLE `product` (
  `product_id` bigint(20) NOT NULL AUTO_INCREMENT,
  `name` varchar(255) NOT NULL,
  `sku` varchar(50) NOT NULL UNIQUE,
  `barcode` varchar(100) DEFAULT NULL,
  `unit_of_measure` varchar(20) DEFAULT 'шт',
  `category_id` bigint(20) DEFAULT NULL,
```

```
  `default_supplier_id` bigint(20) DEFAULT NULL,
  `cost_price` decimal(10,2) DEFAULT 0.00,
  `selling_price` decimal(10,2) DEFAULT 0.00,
  `is_active` tinyint(1) DEFAULT 1,
  `created_at` timestamp NOT NULL DEFAULT current_timestamp(),
  `updated_at` timestamp NOT NULL DEFAULT current_timestamp() ON
UPDATE current_timestamp(),
  PRIMARY KEY (`product_id`),
  KEY `category_id` (`category_id`),
  KEY `default_supplier_id` (`default_supplier_id`),
  CONSTRAINT `product_ibfk_1` FOREIGN KEY (`category_id`)
    REFERENCES `category` (`category_id`) ON DELETE SET NULL,
  CONSTRAINT `product_ibfk_2` FOREIGN KEY (`default_supplier_id`)
    REFERENCES `supplier` (`supplier_id`) ON DELETE SET NULL
)  ENGINE=InnoDB  DEFAULT  CHARSET=utf8mb4
COLLATE=utf8mb4_general_ci;
CREATE TABLE `batch` (
  `batch_id` bigint(20) NOT NULL AUTO_INCREMENT,
  `product_id` bigint(20) NOT NULL,
  `batch_number` varchar(100) NOT NULL,
  `expiry_date` date DEFAULT NULL,
  `production_date` date DEFAULT NULL,
  `cost_price` decimal(10,2) DEFAULT NULL,
  `supplier_batch_id` bigint(20) DEFAULT NULL,
  `quantity` int(11) DEFAULT 0,
  `warehouse_id` bigint(20) DEFAULT NULL,
  `created_at` timestamp NOT NULL DEFAULT current_timestamp(),
  PRIMARY KEY (`batch_id`),
  UNIQUE KEY `unique_batch` (`product_id`,`batch_number`),
```

```
  KEY `warehouse_id` (`warehouse_id`),
  KEY `supplier_batch_id` (`supplier_batch_id`),
  CONSTRAINT `batch_ibfk_1` FOREIGN KEY (`product_id`)
    REFERENCES `product` (`product_id`) ON DELETE CASCADE,
  CONSTRAINT `batch_ibfk_2` FOREIGN KEY (`warehouse_id`)
    REFERENCES `warehouse` (`warehouse_id`) ON DELETE SET NULL,
  CONSTRAINT `batch_ibfk_3` FOREIGN KEY (`supplier_batch_id`)
    REFERENCES `supplier` (`supplier_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4
COLLATE=utf8mb4_general_ci;
DELIMITER $$
CREATE TRIGGER `update_inventory_after_batch_insert` AFTER
INSERT ON `batch`
FOR EACH ROW BEGIN
  IF EXISTS (
    SELECT 1 FROM inventory
    WHERE product_id = NEW.product_id
    AND warehouse_id = NEW.warehouse_id
  ) THEN
    UPDATE inventory
    SET quantity = quantity + NEW.quantity,
      last_updated = NOW()
    WHERE product_id = NEW.product_id
    AND warehouse_id = NEW.warehouse_id;
  ELSE
    INSERT INTO inventory (
      product_id, warehouse_id, quantity,
      min_stock_level, max_stock_level, last_updated
    ) VALUES (
```

```
                NEW.product_id, NEW.warehouse_id, NEW.quantity,
                0, 999999, NOW()
            );
        END IF;
    END$$
DELIMITER $$
```

Листинг Г.1 – SQL-запросы для создания базы данных (фрагменты)

Приложение Д – Словарь данных (таблицы)

Таблица Д.1 – Атрибуты таблицы category (категории товаров)

Поле	Тип данных	Ограничения	Описание
category_id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Уникальный идентификатор категории
name	VARCHAR(255)	NOT NULL	Наименование категории
description	TEXT	NULL	Описание категории
parent_category_id	BIGINT	FOREIGN KEY (category_id), NULL	Идентификатор родительской категории (для иерархии)
created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Дата и время создания записи

Таблица Д.2 – Атрибуты таблицы supplier (поставщики)

Поле	Тип данных	Ограничения	Описание
supplier_id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Уникальный идентификатор поставщика
name	VARCHAR(255)	NOT NULL	Наименование организации-поставщика
contact_person	VARCHAR(255)	NULL	Контактное лицо
email	VARCHAR(255)	NULL	Адрес электронной почты
phone	VARCHAR(50)	NULL	Номер телефона
address	TEXT	NULL	Юридический/фактический адрес
tax_id	VARCHAR(100)	NULL	ИНН/КПП поставщика
rating	DECIMAL(3,2)	CHECK (0-5)	Рейтинг поставщика (от 0 до 5)
created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Дата и время создания записи
updated_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP ON UPDATE	Дата и время последнего обновления

Таблица Д.3 – Атрибуты таблицы warehouse (склады и магазины)

Поле	Тип данных	Ограничения	Описание
warehouse_id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Уникальный идентификатор склада
name	VARCHAR(255)	NOT NULL	Наименование склада/магазина
address	TEXT	NULL	Адрес расположения
type	ENUM	'store', 'warehouse'	Тип: магазин или склад
responsible_person	VARCHAR(255)	NULL	Ответственное лицо
capacity	DECIMAL(12,2)	NULL	Вместимость склада
is_active	TINYINT(1)	DEFAULT 1	Активен ли склад
created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Дата и время создания записи

Таблица В.4 – Атрибуты таблицы product (товары)

Поле	Тип данных	Ограничения	Описание
product_id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Уникальный идентификатор товара

Поле	Тип данных	Ограничения	Описание
name	VARCHAR(255)	NOT NULL	Наименование товара
sku	VARCHAR(50)	NOT NULL, UNIQUE	Артикул товара (Stock Keeping Unit)
barcode	VARCHAR(100)	NULL	Штрихкод товара
unit_of_measure	VARCHAR(20)	DEFAULT 'шт'	Единица измерения
category_id	BIGINT	FOREIGN KEY (category_id), NULL	Категория товара
default_supplier_id	BIGINT	FOREIGN KEY (supplier_id), NULL	Поставщик по умолчанию
cost_price	DECIMAL(10,2)	DEFAULT 0.00	Закупочная цена
selling_price	DECIMAL(10,2)	DEFAULT 0.00	Розничная цена
is_active	TINYINT(1)	DEFAULT 1	Активен ли товар
created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Дата и время создания записи
updated_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP ON UPDATE	Дата и время последнего обновления

Таблица Д.5 – Атрибуты таблицы batch (партии товаров)

Поле	Тип данных	Ограничения	Описание
batch_id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Уникальный идентификатор партии
product_id	BIGINT	FOREIGN KEY (product_id), NOT NULL	Товар
batch_number	VARCHAR(100)	NOT NULL	Номер партии
expiry_date	DATE	NULL	Срок годности
production_date	DATE	NULL	Дата производства
cost_price	DECIMAL(10,2)	NULL	Закупочная цена в данной партии
supplier_batch_id	BIGINT	FOREIGN KEY (supplier_id), NULL	Поставщик партии
quantity	INT	DEFAULT 0	Количество товара в партии
warehouse_id	BIGINT	FOREIGN KEY (warehouse_id), NULL	Склад хранения
created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Дата и время создания записи

Таблица Д.6 – Атрибуты таблицы inventory (остатки товаров)

Поле	Тип данных	Ограничения	Описание
inventory_id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Уникальный идентификатор записи об остатках
product_id	BIGINT	FOREIGN KEY (product_id), NOT NULL	Товар
warehouse_id	BIGINT	FOREIGN KEY (warehouse_id), NOT NULL	Склад
quantity	INT	NOT NULL, DEFAULT 0	Текущее количество товара на складе
min_stock_level	INT	DEFAULT 0	Минимальный уровень запаса
max_stock_level	INT	DEFAULT 1000	Максимальный уровень запаса
last_updated	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP ON UPDATE	Дата и время последнего обновления остатков

Уникальное ограничение: UNIQUE KEY unique_product_warehouse (product_id, warehouse_id) – гарантирует, что товар учитывается на складе только один раз.

Таблица Д.7 – Атрибуты таблицы stockmovement (движения товаров)

Поле	Тип данных	Ограничения	Описание
movement_id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Уникальный идентификатор движения
product_id	BIGINT	FOREIGN KEY (product_id), NOT NULL	Товар
from_warehouse_id	BIGINT	FOREIGN KEY (warehouse_id), NULL	Склад-источник
to_warehouse_id	BIGINT	FOREIGN KEY (warehouse_id), NULL	Склад-назначение
movement_date	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Дата и время движения
type	ENUM	'receipt', 'transfer', 'write_off', 'sale', 'adjustment'	Тип операции
quantity	INT	NOT NULL	Количество перемещаемого товара
reason	TEXT	NULL	Причина перемещения/списания
reference_number	VARCHAR(100)	NULL	Номер документа-основания

Поле	Тип данных	Ограничения	Описание
batch_id	BIGINT	FOREIGN KEY (batch_id), NULL	Партия товара
user_id	BIGINT	FOREIGN KEY (user_id), NOT NULL	Пользователь, совершивший операцию

Таблица Д.8 – Атрибуты таблицы saleshistory (история продаж)

Поле	Тип данных	Ограничения	Описание
sales_id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Уникальный идентификатор продажи
product_id	BIGINT	FOREIGN KEY (product_id), NOT NULL	Проданный товар
warehouse_id	BIGINT	FOREIGN KEY (warehouse_id), NOT NULL	Склад/магазин, с которого продан товар
quantity	INT	NOT NULL	Количество проданного товара
sale_date	DATE	NOT NULL	Дата продажи
sale_price	DECIMAL(10,2)	NOT NULL	Цена продажи
customer_id	BIGINT	NULL	Идентификатор покупателя
transaction_id	VARCHAR(100)	NULL	Идентификатор транзакции

Поле	Тип данных	Ограничения	Описание
batch_id	BIGINT	FOREIGN KEY (batch_id), NULL	Партия товара
created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Дата и время создания записи

Таблица Д.9 – Атрибуты таблицы purchaseorder (заказы поставщикам)

Поле	Тип данных	Ограничения	Описание
order_id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Уникальный идентификатор заказа
supplier_id	BIGINT	FOREIGN KEY (supplier_id), NOT NULL	Поставщик
user_id	BIGINT	FOREIGN KEY (user_id), NOT NULL	Пользователь, создавший заказ
order_date	DATE	NOT NULL	Дата заказа
status	ENUM	'draft', 'ordered', 'delivered', 'cancelled'	Статус заказа
total_amount	DECIMAL(12,2)	DEFAULT 0.00	Общая сумма заказа
expected_delivery	DATE	NULL	Ожидаемая дата поставки
actual_delivery	DATE	NULL	Фактическая дата поставки

Поле	Тип данных	Ограничения	Описание
created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Дата и время создания записи
updated_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP ON UPDATE	Дата и время последнего обновления

Таблица Д.10 – Атрибуты таблицы orderitem (позиции заказа)

Поле	Тип данных	Ограничения	Описание
order_item_id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Уникальный идентификатор позиции заказа
order_id	BIGINT	FOREIGN KEY (order_id), NOT NULL	Заказ
product_id	BIGINT	FOREIGN KEY (product_id), NOT NULL	Товар
quantity	INT	NOT NULL	Количество заказанного товара
unit_price	DECIMAL(10,2)	NOT NULL	Цена за единицу
total_price	DECIMAL(12,2)	GENERATED ALWAYS	Общая стоимость позиции (quantity * unit_price)

Таблица Д.11 – Атрибуты таблицы forecast (прогнозы)

Поле	Тип данных	Ограничения	Описание
forecast_id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Уникальный идентификатор прогноза
product_id	BIGINT	FOREIGN KEY (product_id), NOT NULL	Товар
period	DATE	NOT NULL	Период прогноза
forecast_quantity	INT	NOT NULL	Прогнозируемое количество
calculation_method	VARCHAR(50)	DEFAULT 'moving_average'	Метод расчета прогноза
confidence_level	DECIMAL(5,4)	CHECK (0-1)	Уровень достоверности прогноза
created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Дата и время создания прогноза
actual_quantity	INT	NULL	Фактическое количество (для сверки)

Таблица Д.12 – Атрибуты таблицы user (пользователи системы**)**

Поле	Тип данных	Ограничения	Описание
user_id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Уникальный идентификатор пользователя

Поле	Тип данных	Ограничения	Описание
username	VARCHAR(100)	NOT NULL, UNIQUE	Логин пользователя
password_hash	VARCHAR(255)	NOT NULL	Хеш пароля
role	ENUM	'ADMIN', 'MANAGER', 'STOREKEEPER', 'SENIOR_STOREKEEPER'	Роль пользователя
shopid	BIGINT	FOREIGN KEY (warehouse_id), NULL	Привязка к складу/магазину
full_name	VARCHAR(255)	NULL	Полное имя
email	VARCHAR(255)	NULL	Адрес электронной почты
phone	VARCHAR(50)	NULL	Номер телефона
is_active	TINYINT(1)	DEFAULT 1	Активна ли учетная запись
last_login	TIMESTAMP	NULL	Дата и время последнего входа
created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Дата и время создания учетной записи

Таблица Д.13 – Атрибуты таблицы Stock_alerts

Поле	Тип данных	Ограничения	Описание
alert_id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Уникальный идентификатор уведомления
product_id	BIGINT	NOT NULL	Товар
warehouse_id	BIGINT	NOT NULL	Склад
current_quantity	INT	NOT NULL	Текущее количество товара
alert_type	VARCHAR(50)	DEFAULT 'LOW_STOCK'	Тип уведомления
created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Дата и время создания уведомления
is_resolved	TINYINT(1)	DEFAULT 0	Статус уведомления (решено/не решено)

Таблица Д.13 – Атрибуты таблицы Stock_alerts (уведомления о низком запасе)

Приложение Е – Пояснение к диаграммам проектирования

ER-диаграмма (рисунок А.1) отражает логическую модель данных системы управления запасами розничной сети. При разработке структуры базы данных использовался принцип нормализации для устранения избыточности и обеспечения целостности данных.

Центральными сущностями модели являются product (товары), warehouse (склады/магазины) и supplier (поставщики). Ключевая таблица inventory реализует связь «многие

ко многим» между товарами и складами, храня актуальные остатки и пороговые значения уровня запасов. Таблица batch обеспечивает партионный учёт с контролем сроков годности, что критически важно для управления скоропортящимися товарами.

При реализации в MySQL для всех ключевых связей использованы внешние ключи (FOREIGN KEY), что обеспечивает:

1. Ссылочную целостность – невозможность создания записей, ссылающихся на несуществующие объекты;
2. Каскадное удаление – при удалении товара автоматически удаляются связанные записи в inventory и batch;
3. Защиту от «висячих» ссылок – предотвращение ситуаций, когда запись ссылается на уже удалённый объект.

Автоматизация учётных операций реализована через триггеры: после вставки новой партии в таблицу batch автоматически обновляется соответствующая запись в inventory, а при создании записи о движении товара в stockmovement происходит проверка достаточности остатков и их корректировка. Для сложных операций обновления остатков создана хранимая процедура UpdateInventory, обеспечивающая атомарность транзакций.

UML-диаграмма классов (рисунок 1) представлена в упрощенном виде и отражает многослойную архитектуру приложения, реализованную на базе Spring Boot. Диаграмма включает следующие основные компоненты:

1. JPA-сущности (пакет data.entities) – классы, аннотированные @Entity, которые отображают объектную модель на реляционную структуру базы данных. Каждая сущность соответствует таблице в БД и содержит поля, аннотированные @Column, а также связи @OneToMany, @ManyToOne. Например, класс Supplier содержит поля: supplierId, name, contactPerson, email, phone и т.д.
2. Репозитории (пакет data.repositories) – интерфейсы, расширяющие JpaRepository, предоставляющие стандартные методы CRUD и

возможность определения пользовательских запросов. Репозитории формируют слой доступа к данным, скрывая детали реализации SQL-запросов.

3. DTO (пакет data.dtos) – объекты передачи данных, используемые для обмена информацией между сервисами и контроллерами. DTO позволяют скрыть внутреннюю структуру сущностей и передавать только необходимые поля, например SupplierDto содержит только имя, контактное лицо и email поставщика без служебных полей.

4. Сервисы (пакет buiseness) – классы, реализующие бизнес-логику приложения. Каждый сервис (например, SupplierService) реализует соответствующий интерфейс (SupplierServiceInterface) и содержит методы для выполнения операций, такие как получение списка поставщиков, сохранение изменений, преобразование сущностей в DTO.

5. Контроллеры (пакет controllers) – компоненты, обрабатывающие HTTP-запросы. Контроллеры принимают запросы от клиента, вызывают соответствующие методы сервисов и возвращают данные в формате JSON (для REST-эндпоинтов) или имя HTML-шаблона (для MVC). Например, SupplierController обрабатывает запросы по адресу /api/suppliers/getAllSuppliers и возвращает список поставщиков в виде SupplierDto.

6. Конфигурация (пакет config) – класс SecurityConfig, обеспечивающий настройку безопасности: аутентификацию пользователей, авторизацию на основе ролей (ADMIN, MANAGER, STOREKEEPER), защиту от CSRF-атак и настройку формы логина.

Взаимодействие между слоями организовано по принципу инверсии зависимостей: контроллеры зависят от интерфейсов сервисов, сервисы – от интерфейсов репозиториев, что обеспечивает слабую связанность и тестируемость кода. Сервисы выполняют преобразование сущностей в DTO перед отправкой клиенту, что гарантирует безопасность данных (например, исключение передачи хеша пароля).